

Sistemas Distribuídos

Licenciatura em Engenharia Informática

Trabalho Prático 2

Sistema de Monitorização Ambiental para a Universidade de
Évora



UNIVERSIDADE DE ÉVORA

Miguel Pombeiro, 57829 | Miguel Rocha, 58501

Departamento de Informática
Universidade de Évora
Janeiro 2026

Índice

1	Introdução	2
2	Base de Dados	2
2.1	<i>Devices</i>	2
2.2	<i>Reading</i>	3
3	Escolhas realizadas ao longo do trabalho	3
4	Instruções	5
5	Análise de Performance	6
5.1	Configuração para os testes	6
5.2	Protocolo gRPC	7
5.3	Protocolo MQTT	7
5.4	Protocolo REST	8
5.5	Teste adicional realizado	8
5.6	Análise global dos resultados	8
6	Desafios encontrados	9

1 Introdução

O objetivo deste trabalho é desenvolver e implementar um sistema de monitorização ambiental que suporte a ingestão de dados provenientes de sensores IOT que utilizam diferentes protocolos de comunicação (MQTT, gRPC, REST). A gestão completa dos dispositivos de ser feita através de uma API RESTful, a persistência dos dados deve ser feita com recurso a uma base de dados em PostgreSQL. Desta forma, foram implementados três tipos de simuladores para os três tipos de clientes e um cliente de administração em linha de comandos, para gerir dispositivos e consultar medições. Tal como é pedido, a implementação baseou-se em Java Spring Boot, com JPA/Hibernate.

2 Base de Dados

A persistência dos dados foi feita com recurso a JPA/Hibernate, uma vez que está integrado com o Spring Boot. Através do Hibernate, foi possível mapear de forma clara as entidades aos objetos utilizados em Java e gerir automaticamente os esquemas, sem a necessidade de criar manualmente as tabelas.

Desta forma, foram utilizadas duas entidades principais que permitem representar os dados necessários ao correto funcionamento do sistema:

- **Device**, que representa os vários dispositivos IOT que podem enviar medias ao servidor.
- **Reading**, onde estão inseridas as várias leituras enviadas pelos sensores.

Para além destas, foram ainda utilizadas entidades para a localização dos sensores: **Building**, **Department**, **Floor**, **Room**.

2.1 *Devices*

A entidade **Device** é composta pelos seguintes atributos:

Tabela 1: Representação da entidade **Device**.

Coluna	Tipo	Restrições	Descrição
id	Long	PK, Auto	Identificador único.
protocol	ProtocolType	Not Null	Protocolo de comunicação.
room_id	Long	FK, Not Null	Tabela Room .
department_id	Long	FK, Not Null	Tabela Department .
floor_id	Long	FK, Not Null	Tabela Floor .
building_id	Long	FK, Not Null	Tabela Building .
state	Boolean	Not Null	Estado atual do dispositivo.
created_at	LocalDateTime	Not Null	Data e hora de criação.

2.2 *Reading*

A entidade **Reading** é composta pelos seguintes atributos:

Tabela 2: Representação da entidade **Reading**.

Coluna	Tipo	Restrições	Descrição
id	Long	PK, Auto	Identificador único.
device_id	Long	FK, Not Null	Tabela Device .
temperature	Double	Not Null	Temperatura medida.
humidity	Double	Not Null	Humidade medida.
timestamp	LocalDateTime	Not Null	Data/hora da recolha no sensor.
arrived_timestamp	LocalDateTime	Not Null	Data/hora de chegada ao sistema.

3 Escolhas realizadas ao longo do trabalho

Ao longo do desenvolvimento do sistema, foram tomadas várias decisões. Estas decisões foram importantes para o rumo do projeto, dentre as quais se podem destacar as seguintes:

No que diz respeito à base de dados e à forma de persistência dos dados, foi decidido criar entidades diferentes para os vários níveis de localização (**Building**, **Department**, **Floor**, **Room**), uma vez que foi necessário utilizar identificadores únicos para os vários níveis de localização. Apesar de se tratarem entidades simples, desta forma é possível agregar mais facilmente a pesquisa com base nestes níveis, sendo ainda possível adicionar mais informação no futuro, caso seja necessário.

Ainda no que diz respeito às informações guardadas nas várias tabelas da base de dados, foi decidido incluir em ambas as entidades principais - **Device** e **Reading** -, o *timestamp* da sua criação no servidor. Apesar de este valor da entidade **Device** não ter sido utilizado, este irá permitir uma melhor estruturação e evolução do sistema. Já o valor de *arrived_timestamp*, da entidade **Reading**, foi bastante útil no calculo de medidas de performance, nomeadamente a latência, que irão ser abordadas mais adiante.

Relativamente à implementação dos repositórios, foram realizadas algumas **Queries** que talvez não fossem necessárias realizar, pelo facto de existirem algumas palavras-chave que permitem a conversão automática da pesquisa na base de dados. No entanto, decidimos implementá-las na mesma, porque foi a nossa primeira implementação de algo deste tipo, e para ter mais controlo nas operações realizadas.

Para além dos *endpoints* REST referidos nos enunciado, foi decidida a implementação de alguns outros, nomeadamente para a consulta, por parte do cliente administrador, de estatísticas do sistema e métricas de performance:

- GET `api/devices/stats/total`, que permite visualizar estatísticas dos dispositivos IOT registados no servidor (e.g. total, ativos, total por protocolo).
- GET `api/metrics/raw-level`, que permite obter todas as leituras de um nível de localização especificado, entre um intervalo de tempo, também ele, especificado.
- GET `api/metrics/stats/total`, que permite visualizar estatísticas das leituras registadas no servidor (e.g. total, total por protocolo).
- GET `api/metrics/stats/performance/time`, que permite visualizar medidas de performance do sistema, como a latência dos vários protocolos de comunicação. O intervalo de tempo utilizado no cálculo deve ser providenciado no parâmetro `time`.

Todos estes *endpoints* podem ser consultados através do cliente administrador.

De forma a deixar a implementação o mais modular possível e a fazer alguma separação entre a lógica da persistência de dados e a da comunicação, foram utilizadas DTOs (*Data Transfer Object*). Desta forma, qualquer comunicação que envolvesse trocar dados relativos a estas entidades envolvem um DTO que é depois adequadamente interpretada no destino. Desta forma os modelos da base de dados estão mais protegidos, havendo uma necessidade de conversão interna.

Para a comunicação, nomeadamente com MQTT e gRPC, foram utilizadas as versões dos protocolos fornecidas durante as aulas práticas e não as formas de integração fornecidas pelo Spring Boot. Desta forma foram criados serviços, *beans* e configurações manuais que seguem aquilo que foi explorado nas aulas. Esta decisão prendeu-se com o facto de termos exemplos específicos para cada protocolo, o que nos permitiu explorar melhor a sua implementação, sem que esta estivesse a utilizar a API de configuração do Spring Boot.

Relativamente à implementação do cliente que utiliza o protocolo de comunicação MQTT, decidimos tomar um rumo diferente relativamente ao método de implementação dos outros protocolos. Isto deve-se ao facto de termos implementado vários sensores num único simulador, obrigando a utilização de *threads*. A utilização de *threads* tem um grande impacto na aplicação, já que temos que ter cuidado com os métodos chamados, necessitando de serem *thread safe*. Depois de uma pesquisa, algumas fontes referiam que era *thread safe* enquanto que outras indicavam o contrário, recomendando o uso da classe `MqttAsyncClient`, mas como não foi lecionada decidimos usar a versão lecionada nas aulas práticas. Sendo assim, foi necessário criar uma fila de processamento numa *thread* separada que iria processar as medições uma a uma. Outra opção era ter usado um objeto `MqttClient` por cada sensor, algo que tentamos evitar ao máximo. No fim, esta nossa decisão acabou por ter um grande impacto na *performance* deste protocolo, sendo discutido mais adiante.

Ainda relativamente à implementação dos protocolos de comunicação, mais especificamente do gRPC, foi ainda realizada uma mensagem de envio para o cliente, com uma mensagem a confirmar o sucesso do envio. Apesar de não ter sido implementada no lado do cliente, a ideia principal era efetuar um mecanismo de *retry logic* do lado do cliente, mas não foi implementado devido à falta de tempo, fazendo parte das implementações futuras do projeto.

Por fim, no cliente REST, tal como foi sugerido, foi implementada um processo de *retry logic*, para quando ocorre algum problema no envio de leituras ambientais. Esta implementação foi feita recorrendo à técnica de *linear backoff*, em que é feita uma tentativa em tempos linearmente incrementais até um número máximo de tentativas ou até que o envio seja bem sucedido.

4 Instruções

De modo a utilizar este serviço é necessário fazer os passos descritos adiante, tendo também em conta o conteúdo do ficheiro `application.properties` e os respetivos ficheiros para o docker, `.env` e `docker-compose.yml`. É também necessário ter em conta que ao compilar e executar, devido ao *ProtoBuff* podem ter alguns erros ao executar a primeira vez, de modo a "pegar" as dependências.

1. Inicializar o Docker, usando o comando na diretoria base do projeto

```
| docker compose up
```

2. Conectar a base de dados:

- 2.1. Abrir `http://localhost:16543`

- 2.2. *Login* com as credenciais do ficheiro `.env`

- **Email:** admin@admin.com
- **Password:** admin123

- 2.3. Registrar um novo servidor no pgAdmin:

- **Host name/address:** postgres
- **Port:** 5432
- **Database:** environmentMonitoring
- **Username:** trabalho2
- **Password:** sistemasDistribuidos

3. Na diretoria `server`, compilar e executar o servidor

```
| mvn clean compile  
| mvn spring-boot:run
```

4. Na diretória `admin-cli`, compilar e executar o cliente administrador

```
mvn clean compile
mvn exec:java -Dexec.mainClass="pt.uevora.sdist.monitoring.AdminCLI"
```

5. Na diretória `client-rest`, compilar e executar o cliente rest, que irá simular os dispositivos do protocolo REST

```
mvn clean compile
mvn exec:java -Dexec.mainClass="pt.uevora.sdist.monitoring.ClientRest"
```

6. Na diretória `client-grpc`, compilar e executar o cliente grpc, que irá simular os dispositivos do protocolo gRPC

```
mvn clean compile
mvn exec:java -Dexec.mainClass="pt.uevora.sdist.monitoring.ClientGrpc"
```

7. Na diretória `client-mqtt`, compilar e executar o cliente mqtt, que irá simular os dispositivos do protocolo MQTT

```
mvn clean compile
mvn exec:java -Dexec.mainClass="pt.uevora.sdist.monitoring.ClientMqtt"
```

5 Análise de Performance

O objetivo desta análise de performance é comparar o comportamento do servidor perante as leituras recebidas através dos três protocolos utilizados (MQTT, gRPC, REST), utilizando critérios que refletem tanto a experiência de tempo real, como a capacidade de processamento. Os critérios avaliados são a latência (tempos de processamento/resposta) e o *throughput* (número de leituras processadas por segundo). Pensámos, ainda, em utilizar o número de leituras perdidas como uma métrica de fiabilidade, contudo, como os testes foram executados em ambiente local (*localhost*) e com os recursos partilhados entre o servidor e os simuladores as perdas não são representativas. Desta forma, acabamos por não utilizar esta métrica.

5.1 Configuração para os testes

- Os testes foram executados em máquina local (*localhost*), sendo que tanto os simuladores como o servidor e a base de dados estão a correr na mesma máquina física.
- Todos os clientes têm uma *thread pool* de 4 *threads* para a geração aleatória de leituras.

- Para o teste principal, foram simuladas medições com 100 e 400 sensores em simultâneo para cada protocolo e para o teste secundário 2000 sensores com uma medição única.
- Foi testado um protocolo de cada vez, ou seja, primeiro apenas MQTT, depois apenas gRPC e por fim, apenas REST. Desta forma evitamos a interferência entre protocolos.
- Tentámos minimizar ao máximo outro trabalho que a máquina pudesse estar a fazer, nomeadamente fechar processos não essenciais, desligar sincronizações e mesmo reduzir operações de I/O. Desta forma, tentamos assegurar que as medições fossem realizadas sob condições tão semelhantes quanto possível.
- Testamos configurações iguais em máquinas diferentes, de modo a analisar como as diferenças de hardware e de processamento poderiam afetar os resultados.

5.2 Protocolo gRPC

Com base nos critérios decididos, anteriormente, realizamos as medições da latência, bem como o throughput. Para tal, foi usado o critério de 1 envio de métricas para o servidor por segundo durante 60 segundos, que obteve os seguintes resultados:

Tabela 3: Resultados de desempenho do protocolo gRPC.

Protocolo	Nº sensores	Nº Medições	Latência (ms)	Throughput (RPS)
GRPC	100	6008	2.5301	100.7817
GRPC	100	6008	1.4774	100.5826
GRPC	400	24008	0.1774	401.8681

5.3 Protocolo MQTT

Com base nos critérios decididos, anteriormente, realizamos as medições da latência, bem como o throughput. Para tal, foi usado o critério de 1 envio de métricas para o servidor por segundo durante 60 segundos, que obteve os seguintes resultados:

Tabela 4: Resultados de desempenho do protocolo MQTT.

Protocolo	Nº sensores	Nº Medições	Latência (ms)	Throughput (RPS)
MQTT	100	6020	99.4807	100.7076
MQTT	100	6030	99.3809	100.6426
MQTT	400	24056	184.9172	401.1071

5.4 Protocolo REST

Com base nos critérios decididos, anteriormente, realizamos as medições da latência, bem como o throughput. Para tal, foi usado o critério de 1 envio de métricas para o servidor por segundo durante 60 segundos, que obteve os seguintes resultados:

Tabela 5: Resultados de desempenho do protocolo REST.

Protocolo	Nº sensores	Nº Medições	Latência (ms)	Throughput (RPS)
REST	100	6008	1.5493	100.3206
REST	100	6004	1.0983	100.3325
REST	400	24004	0.1434	400.9086

5.5 Teste adicional realizado

Após a análise do comportamento dos simuladores com o critério de 1 leitura para o servidor por segundo durante 60 segundos, decidiu-se testar o máximo de leituras apenas com uma medição e com um número absurdo de sensores (2000). Este teste foi útil para analisar o *throughput* corretamente, e teve os seguintes resultados:

Tabela 6: Resultados de desempenho com apenas uma medição por dispositivo.

Protocolo	Nº sensores	Nº Medições	Latência (ms)	Throughput (RPS)
GRPC	2000	2000	1.336	1596.1692
MQTT	2000	2000	1413.7635	824.7423
REST	2000	2000	0.7975	1663.8935

5.6 Análise global dos resultados

Após uma detalhada análise dos protocolos, foi possível chegar a uma conclusão inesperada, o protocolo REST foi o que apresentou menor latência e um melhor *throughput*.

Inicialmente a nossa expectativa era de que a comunicação por gRPC seria a mais rápida, uma vez que assenta na troca de mensagens por binário. Já o REST, que assenta em mensagens de texto (tipicamente JSON) deveria apresentar uma latência superior.

O protocolo MQTT apresentou latências extremamente elevadas. Este valores são justificados com a implementação feita para este cliente simulador. Neste caso, como já foi referido, foi utilizado apenas um cliente MQTT que faria o envio das métricas todas dos vários simuladores. Esta decisão, que tentou otimizar o uso do cliente e que se prendeu com a incerteza sobre se o `MqttClient` era ou não *thread safe*, acabou por afetar de

forma muito negativa estes testes. Uma forma de prevenir este efeito seria utilizar um `MqttClient` para cada sensor, simulando o comportamento real, mas que acabou por não ser implementado.

Relativamente aos dados obtidos no teste principal, houve uma diferença entre a primeira e a segunda medição com 100 sensores para cada tipo de comunicação, devido ao facto de ter sido testado esse valor em máquinas diferentes. Achemos que esse teste foi útil, já que as máquinas diferentes apresentam configurações diferentes.

Foi registado um ligeiro aumento no valor de medições testadas, devido à maneira em que os testes foram realizados. No entanto, achamos que não afetou os resultados.

Para o nosso teste principal, os valores para o *throughput* acompanharam o número de sensores, para quantidades de sensores mais baixas. Este facto vai de encontro ao esperado e indica que existe uma capacidade de processamento simultâneo muito elevada por parte do servidor. Assim para o nosso teste adicional, foram feitas medições com uma quantidade de sensores muito superior, que, tal como já descrito acima, permitiu encontrar o *bottleneck* (quantidade de pedidos que é possível processar e enviar em simultâneo) para os diferentes protocolos. Para o REST e gRPC, foi possível verificar que estes valores são relativamente próximos e da ordem dos 1600 readings/s. Já no que diz respeito ao *throughput* para o MQTT, não deve ser feita uma comparação com as restantes tecnologias, uma vez que este tem uma implementação fundamentalmente diferente.

Por fim, apesar destes resultados não corresponderem às nossas expectativas, concluímos que cada um destes protocolos é único sendo necessário analisar outras métricas para clarificar melhor estas diferenças. O resultado do protocolo de comunicação MQTT foi desapontante, mas caracterizou bem a nossa implementação do simulador, sendo necessário considerar para implementações futuras o uso de `MqttAsyncClient`, que eliminaria a necessidade de usar uma fila para tratar do envio com uma *Worker Thread*.

6 Desafios encontrados

Ao longo do desenvolvimento do sistema, o grupo foi encontrando vários desafios que permitiram perceber melhor o trabalho, bem como aprimorar as nossas competências em Java e nas diversas tecnologias utilizadas.

A nossa primeira dificuldade encontrada foi a estudar as tecnologias pedidas no enunciado, ou seja, perceber o tipo de comunicação desta e alguns detalhes importantes que fossem úteis para a implementação do trabalho. Esta dificuldade deveu-se ao facto de serem usadas muitas tecnologias, desde o uso de três protocolos de comunicação diferentes, a um servidor Spring Boot. No entanto, este estudo inicial permitiu-nos ajudar na implementação Java do projeto, tomando um papel bastante importante.

Relativamente ao servidor Spring Boot, também tivemos algumas dificuldades, nomeadamente na integração das tecnologias em todo o projeto, e no uso de algumas anotações específicas, como por exemplo `Beans`, `Service`, `Configuration`. No entanto, o facto de o servidor ser feito em Spring Boot ajudou bastante, já que ele consegue gerir os recursos sozinhos.

As bases de dados criadas com recurso a Hibernate/JPA (Java Persistence API) também representaram um grande desafio, pelo facto de existirem diversas implementações, e por existirem algumas anotações que não conhecíamos e que tivemos que pesquisar.

Relativamente aos protocolos de comunicação, do lado do servidor, algumas implementações também representaram um desafio, nomeadamente o MQTT, e o gRPC, devido ao facto de também existirem versões destes geridas pelo Spring Boot, e às versões utilizadas nas aulas práticas. Já no que diz respeito protocolo REST, este foi o mais fácil de implementar pela sua integração em Spring Boot, com muita documentação. A implementação do MQTT foi a parte mais difícil, para além da sua integração com o Spring Boot, encontramos alguns problemas a conectar com o *Broker* local, mas que rapidamente foram superadas. Na implementação do gRPC, a maior dificuldade foi na utilização do ficheiro proto e das suas características, mas com a documentação das aulas práticas também rapidamente se superou.

Na implementação dos clientes, a dificuldade principal foi tomar atenção às particularidades de todas as tecnologias e preparar os dados na forma correta (pedida pelo servidor), de modo a realizar a ingestão das leituras dos sensores corretamente. Para os vários clientes simuladores, houve a necessidade de utilizar *threads* uma vez que são englobados vários sensores numa única aplicação. Assim, houve a necessidade de verificar se todos os métodos chamados são *thread safe*, algo complicou a implementação do cliente MQTT.

Todos os clientes simuladores, foram implementados com recurso a `ScheduledThreadPool` que nos permitiu controlar melhor os recursos que são utilizados pelos simuladores, bem como o intervalo de tempo em que cada leitura deveria ser enviada ao servidor. Esta implementação também representou um desafio, uma vez que foi necessário ler e perceber a documentação relacionada com `ScheduledExecutorService` e mesmo `ScheduledFuture`.

Para o cliente administrador, o mais difícil foi a realização das tabelas, tendo sido necessário procurar uma implementação eficiente que permitisse reaproveitá-las facilmente para os diversos tipos de clientes, bem como permitir uma boa legibilidade dos dados.

No geral, uma grande dificuldade foi tentar reaproveitar código o mais possível, deixando-o o mais modular e simples possível, que nos facilitou a maioria das nossas implementações.

Depois de superadas as maiores dificuldades, achamos ter conseguido desenvolver um sistema que implementa as funcionalidades pretendidas.